```
"""
app.py
------
Browser-based app for the forex fundamental x technical system.
Launch with:  streamlit run app.py
(or just double-click run_app.command / run_app.bat)

This is a UI wrapper only -- all the actual scoring logic lives in
fundamental.py, technical.py, data_feed.py, fred_fetch.py, unchanged
from the command-line version, so nothing about how numbers are
computed is different here.
"""

import os
import tempfile

import pandas as pd
import streamlit as st

from fundamental import load_fundamental_data, compute_currency_strength, WEIGHTS
from technical import compute_technical_signals
from data_feed import get_ohlc, generate_synthetic_ohlc
from engine import default_pairs
import fred_fetch

st.set_page_config(page_title="Forex Fundamental x Technical", layout="wide")

TEMPLATE_CSV = os.path.join(os.path.dirname(__file__), "fundamental_data_template
TEMPLATE_COLUMNS = fred_fetch.TEMPLATE_COLUMNS

if "fundamentals_df" not in st.session_state:
    st.session_state.fundamentals_df = pd.read_csv(TEMPLATE_CSV).set_index("Curre
if "results" not in st.session_state:
    st.session_state.results = None

st.title("Forex: Fundamental x Technical Cross-Reference")
st.caption("Screening tool for idea generation -- not a signal generator. "
           "See the README for the full list of limitations.")

# ---------------------------------------------------------------------
# SIDEBAR: data sources & settings
# ---------------------------------------------------------------------
with st.sidebar:
    st.header("1. Fundamental data")
```

```
    source = st.radio("Source", ["FRED (auto-fetch)", "Upload CSV", "Edit in-app"

    if source == "FRED (auto-fetch)":
        fred_key = st.text_input("FRED API key", type="password",
                                 value=os.environ.get("FRED_API_KEY", ""),
                                 help="Free key: https://fredapi.stlouisfed.org/
        default_currencies = list(fred_fetch.CURRENCY_TO_COUNTRY.keys())
        pick_currencies = st.multiselect("Currencies", default_currencies, defaul
        if st.button("Fetch from FRED", type="primary", disabled=not fred_key):
            with st.spinner(f"Fetching {len(pick_currencies)} currencies from FRE
                try:
                    fetched = fred_fetch.build_fundamental_dataframe(fred_key, pi
                    st.session_state.fundamentals_df = fetched
                    st.success("Fetched. Review/edit below before running the ana
                except Exception as e:
                    st.error(f"Fetch failed: {e}")

    elif source == "Upload CSV":
        uploaded = st.file_uploader("Fundamental data CSV", type="csv")
        if uploaded is not None:
            try:
                st.session_state.fundamentals_df = pd.read_csv(uploaded).set_inde
                st.success("Loaded.")
            except Exception as e:
                st.error(f"Could not read CSV: {e}")

    st.divider()
    st.header("2. Price data")
    use_synthetic = st.toggle("Offline demo mode (synthetic prices, no internet n
    period = st.selectbox("Lookback period", ["3mo", "6mo", "1y", "2y"], index=1)
    interval = st.selectbox("Interval", ["1d", "1h"], index=0)

    st.divider()
    st.header("3. Pairs")
    currencies_available = list(st.session_state.fundamentals_df.index)
    auto_pairs = default_pairs(currencies_available)
    pairs_text = st.text_area("Pairs (comma-separated)", value=",".join(auto_pair

# ---------------------------------------------------------------------
# MAIN: editable fundamental data table
# ---------------------------------------------------------------------
st.subheader("Fundamental data (editable)")
st.caption("Tweak any value directly in the table -- the analysis below uses what

edited = st.data_editor(
    st.session_state.fundamentals_df.reset_index(),
    num_rows="dynamic",
```

```
    use_container_width=True,
    key="fundamentals_editor",
)
st.session_state.fundamentals_df = edited.set_index("Currency")

with st.expander("Metric weights (fundamental.py WEIGHTS)"):
    st.write(pd.Series(WEIGHTS, name="weight").to_frame())
    st.caption("To change these, edit the WEIGHTS dict at the top of fundamental.

st.divider()

# ---------------------------------------------------------------------
# RUN
# ---------------------------------------------------------------------
if st.button("Run analysis", type="primary"):
    df = st.session_state.fundamentals_df.copy()
    pairs = [p.strip().upper() for p in pairs_text.split(",") if p.strip()]

    with tempfile.NamedTemporaryFile(mode="w", suffix=".csv", delete=False) as tm
        df.reset_index().to_csv(tmp.name, index=False)
        tmp_path = tmp.name

    try:
        raw = load_fundamental_data(tmp_path)
        strength = compute_currency_strength(raw)
        max_abs = strength["CompositeScore"].abs().max() or 1.0
        norm_strength = strength["CompositeScore"] / max_abs

        rows = []
        progress = st.progress(0.0, text="Evaluating pairs...")
        for i, pair in enumerate(pairs):
            base, quote = pair[:3], pair[3:]
            if base not in norm_strength.index or quote not in norm_strength.inde
                st.warning(f"Skipping {pair}: missing fundamental data for {base}
                continue
            fundamental_bias = norm_strength[base] - norm_strength[quote]

            try:
                if use_synthetic:
                    seed = abs(hash(pair)) % (2**32)
                    ohlc = generate_synthetic_ohlc(n=220, seed=seed)
                else:
                    ohlc = get_ohlc(pair, period=period, interval=interval)
                tech = compute_technical_signals(ohlc)
            except Exception as e:
                st.warning(f"Skipping {pair}: price data error ({e})")
                continue
```

```
            technical_score = tech["TechnicalScore"]
            agree = (fundamental_bias > 0) == (technical_score > 0) and fundament
            conviction = abs(fundamental_bias) * abs(technical_score)
            if not agree:
                conviction *= 0.3

            rows.append({
                "Pair": pair, "Base": base, "Quote": quote,
                "FundamentalBias": round(float(fundamental_bias), 3),
                "TechnicalScore": technical_score,
                "Structure": tech["Structure"],
                "Agreement": bool(agree),
                "ConvictionScore": round(float(conviction), 3),
                "RSI14": tech["RSI14"],
            })
            progress.progress((i + 1) / max(len(pairs), 1), text=f"Evaluated {pai

        progress.empty()
        st.session_state.results = {
            "strength": strength,
            "pairs": pd.DataFrame(rows).sort_values("ConvictionScore", ascending=
            if rows else pd.DataFrame(),
        }
    finally:
        os.unlink(tmp_path)

# ---------------------------------------------------------------------
# RESULTS
# ---------------------------------------------------------------------
if st.session_state.results is not None:
    strength = st.session_state.results["strength"]
    pairs_df = st.session_state.results["pairs"]

    tab1, tab2, tab3 = st.tabs(["Currency Strength", "Pair Cross-Reference", "Cle

    with tab1:
        st.bar_chart(strength["CompositeScore"])
        st.dataframe(strength[["CompositeScore", "Rank"]], use_container_width=Tr
        st.download_button("Download strength ranking (CSV)",
                           strength.to_csv().encode(), "currency_strength.csv")

    with tab2:
        if pairs_df.empty:
            st.info("No pairs were evaluated -- check the Pairs list and fundamen
        else:
            st.dataframe(pairs_df, use_container_width=True)
```

```
            st.download_button("Download pair results (CSV)",
                               pairs_df.to_csv(index=False).encode(), "pair_cros

    with tab3:
        if pairs_df.empty:
            st.info("No pairs were evaluated.")
        else:
            top = pairs_df[pairs_df["Agreement"]].head(5)
            if top.empty:
                st.info("No pairs currently show fundamental/technical agreement.
            for _, r in top.iterrows():
                direction = "Bullish" if r["FundamentalBias"] > 0 else "Bearish"
                st.metric(
                    label=f"{r['Pair']} -- {direction} {r['Base']} vs {r['Quote']
                    value=f"Conviction {r['ConvictionScore']:.2f}",
                    delta=f"structure: {r['Structure']}",
                )
else:
    st.info("Set your data sources in the sidebar, review the fundamental table a
```